Módulo: PSP. Grupo: 2.º DAM	**Actividad:** Examen Práctico T1 (Var. C)	**Fecha:** ____/____/2026

Alumno/a: _____

_____

Programación de Servicios y Procesos (RA1 – RA2 – RA3)

Prueba escrita. No se utilizará el ordenador durante la resolución; al finalizar se permitirá comprobar el funcionamiento del código. Se facilita por separado una hoja de consulta de APIs. Se valoran la coherencia de las clases, el empleo correcto de los métodos y la gestión de errores, no la memorización de importaciones.

1. Programación multiproceso. Auditoría de recibos (2,5 puntos)

Crea una aplicación Java con dos clases, `AuditorRecibos` y `LanzadorAuditoria`. El objetivo es contabilizar cuántos números de recibo dentro de un rango terminan en el dígito 7.

- `AuditorRecibos` **(Proceso hijo):** Recibirá por argumentos de `main(String[] args)` dos números enteros positivos correspondientes a los límites de una serie de recibos. Recorrerá el intervalo (ambos extremos incluidos), contará cuántos números terminan en 7 y mostrará el total por la salida estándar mediante `System.out.println()`.
- `LanzadorAuditoria` **(Proceso padre):**
  - Definirá dos rangos de auditoría (por ejemplo, 1000–5000 y 5001–9000).
  - Creará dos procesos hijos mediante `ProcessBuilder` de forma concurrente, sin esperar a que finalice el primero para lanzar el segundo.
  - Recuperará la salida de cada hijo abriendo flujos con `getInputStream()`, `InputStreamReader` y `BufferedReader`.
  - Esperará obligatoriamente la finalización de ambos mediante `waitFor()` y sumará los resultados, mostrando el recuento de cada hijo y el total global.

Requisito: Controla entradas no válidas y excepciones (`IOException`, `InterruptedException`). Añade comentarios breves que expliquen la comunicación y la espera.

2. Sockets TCP y multihilo. Cajero Automático (4,5 puntos)

Desarrolla una aplicación Java con un servidor TCP y dos clientes independientes: `TerminalIngresos` y `AppCliente`. El servidor escuchará en `127.0.0.1:8000` y atenderá a ambos clientes asignando un hilo a cada uno.

- **Clase `GestorSaldo` (Monitor):** Guarda una cantidad entera `saldoMovimiento` y un booleano `operacionPendiente`. Sus métodos `synchronized` utilizarán `wait()` y `notifyAll()` para garantizar que el terminal no sobrescriba un ingreso sin procesar, y que la app espere si todavía no hay un nuevo ingreso validado.
- **Cliente `TerminalIngresos`:** Establece una conexión TCP con el servidor. Envía secuencialmente cinco cantidades enteras a ingresar (valores entre 10 y 500), enviando una por línea, y finaliza. (Se puede usar `Random`).
- **Cliente `AppCliente`:** Establece una segunda conexión TCP. Envía cinco peticiones con la palabra «CONSULTAR» y muestra por consola los cinco ingresos validados que le devuelve el servidor, uno por línea.
- **Servidor `ServidorBanco`:** Crea un `ServerSocket`, acepta las dos conexiones entrantes de manera secuencial y delega el trabajo en un `HiloIngresos` y un `HiloApp` que comparten el mismo objeto `GestorSaldo`. El primer hilo lee los datos del terminal y los almacena; el segundo espera peticiones «CONSULTAR», recupera la medida del monitor y la responde a la App.

Requisito: Emplea `BufferedReader` y `PrintWriter` (con auto-flush). Asigna prioridades distintas a los hilos, inícialos con `start()`, consulta `isAlive()`, asegúralos con `join()` y cierra todos los sockets.

3. Biblioteca java.util.concurrent. Cola de Transferencias (3 puntos)

Simula un sistema de transferencias interbancarias utilizando la librería de concurrencia avanzada:

- Configura un `ExecutorService` con un pool fijo de tres hilos.
- Comparte entre las tareas una `ArrayBlockingQueue<Integer>` con capacidad 5 y un contador atómico `AtomicInteger transferenciasCompletadas` inicializado a 0.
- **1 Productor:** Introduce 20 identificadores de transferencia (del 1 al 20) utilizando `put()`. Al terminar, introduce dos marcas de fin con valor `-1` (una para cada hilo consumidor).
- **2 Consumidores:** Extraen identificadores mediante `take()`. Si extraen `-1`, terminan. Si es una transferencia normal, imprimen «Transferencia procesada: ID» e incrementan el contador usando `incrementAndGet()`.
- **Principal:** Entrega las tareas al pool, llama a `shutdown()`, bloquea con `awaitTermination()` y muestra el contador final (debe ser 20).

Cuestiones teóricas básicas

Integradas en la calificación de los apartados de los ejercicios 1 y 2 correspondientes.

A. ¿Qué significa ejecutar tareas de forma concurrente? Indica un ejemplo útil.

B. Distingue ejecución paralela de ejecución distribuida.

C. Explica qué representa el PID de un proceso y cita dos estados posibles.

D. ¿Qué recursos de memoria comparten dos hilos de un mismo proceso que dos procesos independientes no comparten directamente?

E. ¿Por qué llamar directamente a `run()` no equivale a utilizar `start()` ? Cita dos estados de un hilo.

F. ¿Qué indica la prioridad de un hilo? ¿Permite asegurar cuál se ejecuta primero?

G. Explica qué es una conexión mediante sockets TCP y qué función cumple cada extremo, cliente y servidor.

H. ¿Qué hacen `accept()` en ServerSocket y `getInputStream()` en Socket?

Ej.	Evidencia exigida	Puntos
1	`AuditorRecibos` : argumentos, recuento y salida (1.1) + `LanzadorAuditoria` : dos procesos (1.2)	0,90
1	Lectura flujos, `waitFor()` y totales (1.3) + Errores y validación (1.4) + Teoría (A-D)	1,60
2	`GestorSaldo` : valor, estado y monitor (2.1) + Sockets Clientes: conexión y envíos (2.2, 2.3)	1,95
2	`ServidorBanco` : `accept()` , hilos y flujos (2.4) + Coordinación, prioridades (2.5)	1,70
2	Documentación y excepciones (2.6) + Cuestiones Teóricas (E-H)	0,85
3	Cola compartida (3.1) + Productor (3.2) + Consumidores (3.3) + Ejecutor (3.4)	3,00
	CALIFICACIÓN TOTAL	**10,00**